Part I: Background notes

Supporting material for understanding the paper. Not presented.

---

1 Chain rule

Setup. Let $h : \mathbb{R}^n \to \mathbb{R}^m$ be one layer and $\ell : \mathbb{R}^m \to \mathbb{R}$ be everything after it (the rest of the network and the loss), so the loss is $L(x) = \ell\big(h(x)\big)$. The *local Jacobian* of the layer is the $m \times n$ matrix

$$J_h(x) = \frac{\partial h}{\partial x}, \qquad (J_h)_{ij} = \frac{\partial h_i}{\partial x_j}.$$

Chain rule.

$$\nabla_x L \;=\; J_h(x)^\top \, \nabla \ell\big(h(x)\big).$$

Here $\nabla \ell\big(h(x)\big)$ is the *downstream gradient* (what the rest of the network wants the layer's output to do), and $J_h(x)^\top$ converts it into a gradient on the layer's input x.

Intuition. The downstream gradient says how the loss would change if the layer's *output* moved. The local Jacobian says how the layer's output moves when its *input* x moves. Multiplying the two gives how the loss changes when x moves, which is what gradient descent needs to update x. Backpropagation repeats this layer by layer, so every layer's local Jacobian is a factor in the gradient that reaches earlier layers.

Two cases relevant to this paper.

1. **Local Jacobian is zero**, $J_h(x) = 0$. Then

$$\nabla_x L = 0 \quad \text{for every downstream gradient.}$$

Meaning: small changes to x do not change the layer's output, so the loss gives x no direction to move. Gradient descent never updates x, and nothing before this layer learns either. This is what rounding (quantization) does.

2. **Local Jacobian is the identity**, $J_h(x) = I$. Then

$$\nabla_x L = \nabla \ell\big(h(x)\big).$$

Meaning: the downstream gradient is passed back unchanged, as if the layer did nothing. Whatever the layer actually did to x (for example, adding a rounding error) has no effect on the gradient, so earlier layers get no signal about it. This is what the Straight-Through Estimator (STE) assumes.

2 Ridge regression

Why needed. The paper's dequantizer g is a ridge-regression fit of the original x from the rounded q (with $u = q$, $v = x$ below).

Notation. For $u, v \in \mathbb{R}^N$: $\bar{u} = \frac{1}{N}\sum_i u_i$, $\overline{uv} = \frac{1}{N}\sum_i u_i v_i$, $\mathrm{Var}_u = \overline{u^2} - \bar{u}^2$, $\mathrm{Cov}_{uv} = \overline{uv} - \bar{u}\,\bar{v}$, and $\mathbf{1}$ is the all-ones vector.

Problem. Fit $v \approx a\,u + c\mathbf{1}$, penalizing only the slope:

$$\min_{a,c} \ \frac{1}{2N}\|a\,u + c\mathbf{1} - v\|^2 + \frac{\lambda}{2}a^2, \qquad \lambda > 0.$$

Setting derivatives to zero:

$$c^* = \bar{v} - a^*\bar{u}, \qquad a^* = \frac{\mathrm{Cov}_{uv}}{\mathrm{Var}_u + \lambda}.$$

Without intercept ($c = 0$): $a^* = \dfrac{\overline{uv}}{\overline{u^2} + \lambda}$.

Fit. The vector of the line's predictions at the data points:

$$\text{fit} = a^*u + c^*\mathbf{1} = a^*(u - \bar{u}\mathbf{1}) + \bar{v}\mathbf{1}.$$

Role of λ. (i) *Stability:* the denominator is at least $\lambda > 0$, so no division by zero even if all u_i are equal. (ii) *Shrinkage:* as $\lambda \to \infty$, $a^* \to 0$ and the fit becomes the mean $\bar{v}\mathbf{1}$.

3 Quantization

In short. We select a grid of integers, assign one integer to each of our real numbers (rescale, then round to the nearest), and then try to get our real numbers back from those integers. Only the integers (plus one s_f, b_f per block) are stored; the recovery is approximate, and the gap is the rounding error. The paper keeps the first two steps and changes only the last one: how to get back from the integers to the best possible real numbers.

Setup. A block $x \in \mathbb{R}^N$ (weights or activations) is mapped to a k-bit integer grid $\{q_{\min}, \ldots, q_{\max}\}$ with 2^k levels. Notation AaWw: a-bit activations, w-bit weights.

Step 1: prequantization f. Rescale x onto the grid's range.

- Linear (centred data, e.g. weights): $f(x) = \dfrac{x}{s_f}, \quad s_f = \dfrac{\max_i |x_i|}{q_{\max}}$.
- Affine (uncentred data, e.g. ReLU activations): $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}, \quad s_f = \dfrac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad b_f = x_{\min} - s_f\, q_{\min}$.

Step 2: rounding and the error. round is nearest integer, entrywise.

$$q = \text{round}\big(f(x)\big) = f(x) + \delta, \qquad \delta = \text{round}\big(f(x)\big) - f(x), \qquad |\delta_i| \le \tfrac{1}{2}.$$

Step 3: standard dequantization. Undo the rescaling ($b_f = 0$ in the linear case):

$$\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta.$$

Quantization is therefore x plus an additive error of size at most $s_f/2$ per entry; fewer bits means larger s_f, hence larger error.

4 The problem: zero gradient and the Straight-Through Estimator

The chain during training. The network stores the real x; the next layer uses the quantized-then-dequantized $\hat{x}$:

$$x \xrightarrow{\ f\ } f(x) \xrightarrow{\ \text{round}\ } q \xrightarrow{\ s_f q + b_f \mathbf{1}\ } \hat{x} \longrightarrow \text{rest of network} \longrightarrow L,$$

where

- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$ rescales x onto the integer grid ($b_f = 0$ in the linear case); s_f is the step size of the grid in real units, b_f the real value of integer 0;
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$ is the stored integer vector, δ the rounding error, $|\delta_i| \le \frac{1}{2}$;
- $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$ is x as seen after quantization.

Chain rule. Gradient descent needs $\nabla_x L$:

$$\nabla_x L = \Big(\frac{\partial \hat{x}}{\partial x}\Big)^{\top} \nabla_{\hat{x}} L, \qquad \frac{\partial \hat{x}}{\partial x} = \underbrace{s_f}_{\text{scale back}} \cdot \underbrace{\frac{\partial q}{\partial f(x)}}_{\text{round}} \cdot \underbrace{\frac{1}{s_f}}_{f} = \frac{\partial q}{\partial f(x)}.$$

The two scalings cancel, so the local gradient is exactly the slope of round.

Zero gradient. round is a staircase: $\frac{d}{du}\,\text{round}(u) = 0$ for $u \notin \mathbb{Z} + \frac{1}{2}$ (undefined at half-integers). Hence $\frac{\partial q}{\partial f(x)} = 0$ almost everywhere and $\nabla_x L = 0$: x and all earlier layers never update (case 1 of Section 1).

STE (Bengio et al., 2013). Treat the rounding error as a constant in the backward pass:

$$q = f(x) + \delta, \qquad \delta = \text{stopgrad}\big(\text{round}(f(x)) - f(x)\big).$$

Forward: $q = \text{round}(f(x))$ exactly. Backward: $\frac{\partial q}{\partial f(x)} = I$, so $\frac{\partial \hat{x}}{\partial x} = I$ and $\nabla_x L = \nabla_{\hat{x}} L$ (case 2 of Section 1).

The blind spot.

$$\text{forward:}\quad \hat{x} = x + s_f\,\delta, \qquad\qquad \text{backward:}\quad \frac{\partial \hat{x}}{\partial x} = I.$$

The forward pass is changed by the error, but the local Jacobian is the same as with no quantization at all; δ does not appear in it. Earlier layers get no signal about how their rounding error hurts the loss, so they cannot learn to be robust to it. At 1–2 bits $s_f\delta$ is large and STE training becomes unstable. (δ still affects *where* $\nabla_{\hat{x}} L$ is evaluated; it is absent only from the local factor $\partial\hat{x}/\partial x$, which is the factor the paper changes.)

Part II: Slides

Why quantization, and the forward pass during training

Why quantization. Modern neural networks are huge: a language model has billions of weights, each stored as a 32-bit or 16-bit floating-point number. This makes models too big to fit on a phone, slower to run, and costly in energy. Quantization stores each weight with far fewer bits: 4, 2 or 1 bit instead of 16 or 32.

Training with quantization. During training the network keeps the float weights x and updates them by gradient descent. We could compute with x, but we deliberately use the quantized version $\hat{x}$ in the forward pass, so the network learns to be robust to the rounding it will face at inference. After training, x is discarded; only the low-bit integers q (plus one s_f, b_f per block) are kept.

The forward pass during training.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{s_f q + b_f \mathbf{1}} \hat{x} \longrightarrow \text{rest of network} \longrightarrow L$$

- $x \in \mathbb{R}^N$: float block, kept and updated during training.
- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$: rescales x onto the k-bit integer grid; s_f = grid step in real units, b_f = real value of integer 0 ($b_f = 0$ in the linear case).
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$: the integers, with rounding error $|\delta_i| \le \frac{1}{2}$.
- $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$: x as seen after quantization; this is what the next layer uses.

The problem: the true gradient is zero

Chain rule through the quantizer. Gradient descent needs $\nabla_x L$:

$$\nabla_x L = \Big(\frac{\partial \hat{x}}{\partial x}\Big)^\top \nabla_{\hat{x}} L, \qquad \frac{\partial \hat{x}}{\partial x} = \underbrace{s_f}_{\text{scale back}} \cdot \underbrace{\frac{\partial q}{\partial f(x)}}_{\text{round}} \cdot \underbrace{\frac{1}{s_f}}_{f} = \frac{\partial q}{\partial f(x)}.$$

The two scalings cancel: the local gradient is exactly the slope of round.

Round is a staircase.

$$\frac{d}{du}\,\mathrm{round}(u) = 0 \quad \text{for } u \notin \mathbb{Z} + \tfrac{1}{2}, \qquad \text{undefined at } u \in \mathbb{Z} + \tfrac{1}{2}.$$

Hence $\dfrac{\partial q}{\partial f(x)} = 0$ almost everywhere, so

$$\nabla_x L = 0.$$

Consequence. A small change in x does not change q, so the loss gives x no direction to move: x and every layer before it never update. Training cannot start.

The standard fix: Straight-Through Estimator (STE)

Idea. Forward pass: round exactly as before. Backward pass: pass the gradient straight through the rounding, i.e. replace the true slope of round (zero) by 1. This is a chosen rule, not a derived derivative (Bengio et al., 2013).

Decomposition. Write the rounded value as the smooth part plus the rounding error:

$$q = \text{round}\big(f(x)\big) = f(x) + \delta, \qquad \delta = \text{stopgrad}\Big(\text{round}\big(f(x)\big) - f(x)\Big),$$

where stopgrad means: same value in the forward pass, treated as a constant (zero derivative) in the backward pass.

Forward. $f(x) + \delta = \text{round}\big(f(x)\big)$ exactly, so the network still sees quantized values.

Backward. δ is a constant, so

$$\frac{\partial q}{\partial f(x)} = \frac{\partial f(x)}{\partial f(x)} + \underbrace{\frac{\partial \delta}{\partial f(x)}}_{=0} = I.$$

Substituting into the chain rule of the previous slide,

$$\frac{\partial \hat{x}}{\partial x} = s_f \cdot I \cdot \frac{1}{s_f} = I, \qquad \text{hence} \qquad \nabla_x L = \nabla_{\hat{x}} L.$$

Effect. The gradient reaching x is no longer zero, so x and earlier layers update and training works.